

试题：FPGA 布线算法

FPGA（Field-Programmable Gate Array，可编程逻辑门阵列）的布线问题是指在电路系统设计中使用时，需要将FPGA逻辑电路的各个元件（如逻辑门、寄存器等）通过内部的可编程连线资源连接起来，以实现所需的功能。这个过程涉及到在FPGA的可编程逻辑单元中进行逻辑资源的配置和连接，以及在可编程互连资源（如可编程开关矩阵）中进行连线的选择和设置。本试题主要围绕如何利用可编程互连资源完成多个线网的布线展开。

FPGA 是一种可编程的集成电路，其基本结构包括以下几个主要组成部分：

- 可编程逻辑单元（PL，Programmable Logic）：由大量的逻辑单元（Look-Up Tables，LUTs）和寄存器组成，可以通过编程方式配置为不同的逻辑功能，实现各种数字电路的功能。LUTs 通常用于实现逻辑运算和存储逻辑状态，而寄存器用于存储和处理数据。
- 可编程互连资源：可编程互连资源是用于连接可编程逻辑单元之间的内部连线资源。它通常由金属线和可编程开关矩阵（SM，Switch Matrix）组成，通过配置开关矩阵，可以实现不同 PL 之间的连线。可编程互连资源的灵活性使得 FPGA 能够根据设计需求动态地建立各个 PL 之间的连接。

FPGA 的布线通常是在 PL 配置完成以后，有效地利用可编程互连资源进行布线，在满足设计功能和性能要求的前提下，完成各个 PL 之间的连接，以达到最优的布线结果。在布线过程中，一般需要考虑如下几个关键问题：

- 连线冲突：由于 FPGA 互连资源有限，不同逻辑元件可能需要使用相同的连线资源，导致冲突。布线时需要解决这些冲突，确保每个元件得到正确的连线，保证布通率。
- 时序约束：FPGA 中的逻辑电路需要满足时序约束，即信号在特定的时钟周期内到达目标点。布线过程需要考虑时序约束，确保布线后的电路能够满足时序要求，避免时钟延迟和时序错误。
- 优化目标：布线过程中可能存在多个优化目标，如最短路径、最小延迟、最小功耗等。布线算法需要考虑这些目标，并根据设计需求进行优化。

布线问题是 FPGA 设计中的重要环节，它直接影响到电路的性能、可靠性和资源利用率。因此，针对不同的设计需求和约束条件，需要选择适当的布线算法和策略来解决 FPGA 的布线问题。

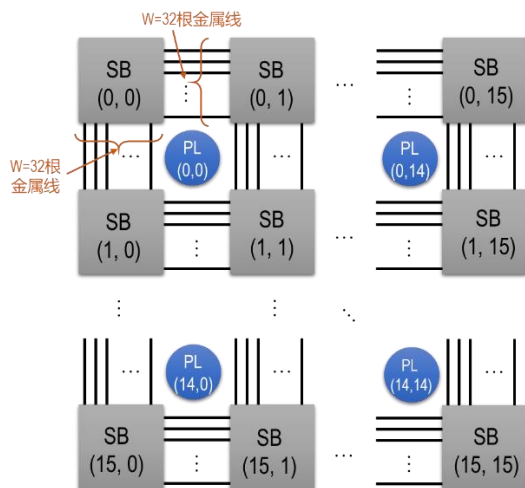


图 1、一个简单 FPGA 的硬件结构示意图。

1. 电路结构介绍：

图 1 定义了一个简单的 FPGA 硬件结构, 包含 15×15 个 PL 和 16×16 个可编程互连模块 (SB, Switch Block)。如图 1 所示, PL、SB 各自的相对位置可以用二维坐标 (x, y) 表示。每两个 SB 之间有 $W = 32$ 个金属互连线 (可记为 $\{m_i; i = 0, 1, 2, \dots, 31\}$), 构成一组单倍线。每一个 SB 中都包含可配置的 SM, 通过配置 SB (实际上是配置 SB 中的 SM), 可以实现 SB 上下左右四个方向上不同金属线之间的连接。

这里我们将每组单倍线分成 4 个子集, 记为 $s_0 = \{m_i; i = 0, 1, 2, \dots, 7\}$, $s_1 = \{m_i; i = 8, 9, \dots, 15\}$, $s_2 = \{m_i; i = 16, 17, \dots, 23\}$, $s_3 = \{m_i; i = 24, 25, \dots, 31\}$ 。假定每个 PL 有 4 个 IO (记为 $\{IO_i; i = 0, 1, 2, 3\}$)。通过对 PL 进行配置, 可以令每一个 IO_i 分别连接上下左右 4 组单倍线中 4 个对应子集 s_i 中的任意且唯一一根金属线 (即 IO_i 可以连接到 4 个对应子集中的任意一根金属线; 但一旦连接后, 就不能够再连接其他金属线了), 这里 $i \in \{0, 1, 2, 3\}$ 。例如, 如图 2 所示, 通过配置 PL, 可以使得其内部的 IO_0 连接到 s_0^{up} 、 s_0^{right} 、 s_0^{down} 和 s_0^{left} (这里的上标表示相对 PL 的位置) 这 4 个子集中的任意且唯一一根金属线, 同时 IO_0 永远不能连接 s_i^{up} 、 s_i^{right} 、 s_i^{down} 和 s_i^{left} ($i \in \{1, 2, 3\}$) 中的任意金属线。同样的, 通过配置 PL, 可以使得 IO_1 连接到 s_1^{up} 、 s_1^{right} 、 s_1^{down} 和 s_1^{left} 这 4 个子集中的任意唯一金属线, IO_2 连接到 s_2^{up} 、 s_2^{right} 、 s_2^{down} 和 s_2^{left} 这 4 个子集中的任意唯一金属线, IO_3 连接到 s_3^{up} 、 s_3^{right} 、 s_3^{down} 和 s_3^{left} 这 4 个子集中的任意唯一金属线。

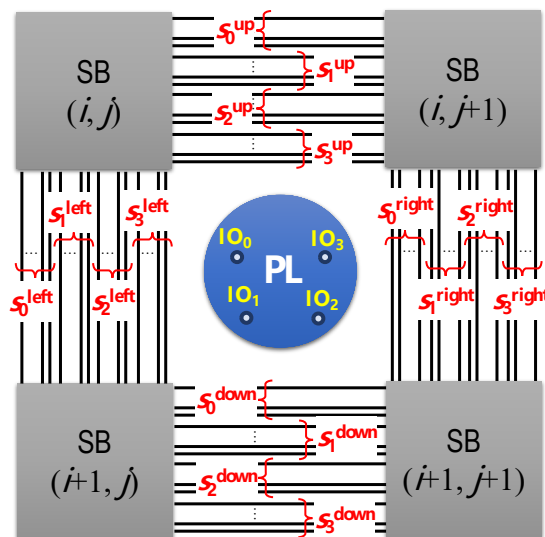


图 2、一个 PL 及其周围 4 个 SB 和 4 组单倍线示意图。

图 3 给出了 SB 的互连配置方式示意图。同样的, 我们将接入 SB 的四个方向上的四组单倍线分别划分为四个子集 s_0, s_1, s_2, s_3 , 这里对应的上标表示金属线相对于 SB 的位置。假定由于硬件资源的限制, 通过配置 SB, 我们能够实现的不同方向金属线之间的连接关系规则如下:

- 任意一根互连金属线在将信号传入 SB 后, 可以通过配置 SB, 将信号从其他三个方向对应子集的金属线输出, 即一个输入 SB 的信号最多可以同时从三个方向输出。例如, 如图 3 所示, 从左边 s_0^{left} 进入 SB 的信号, 可以通过配置, 从上方 s_1^{up} 、右边 s_0^{right} 和/或下方 s_3^{down} (这三个子集是 s_0^{left} 在不同方向上的对应子集) 中的任一金属线输出, 但是不能再从左边 s_1^{left} 、 s_2^{left} 或者 s_3^{left} 输出。
- 假定由于 SB 模块硬件设计的限制, 任意两方向的金属线子集间可以通过配置 SB 实现的互连关系由如下六个子集对定义:

- a) $(s_i^{\text{left}}, s_i^{\text{right}})$: 例如, 如图 3 所示, 当 $i=0$ 时, 通过配置 SB, 可以使得 s_0^{left} 子集中的金属线与 s_0^{right} 子集中的金属线通过一定规则连接。
- b) $(s_i^{\text{left}}, s_{(i+1) \bmod 4}^{\text{up}})$: 例如, 如图 3 所示, 当 $i=0$ 时, 通过配置 SB, 可以使得 s_0^{left} 子集中的金属线与 $s_{(0+1) \bmod 4}^{\text{right}} = s_1^{\text{right}}$ 子集中的金属线通过一定规则连接。
- c) $(s_i^{\text{down}}, s_{(i+1) \bmod 4}^{\text{left}})$: 例如, 如图 3 所示, 当 $i=3$ 时, 通过配置 SB, 可以使得 s_3^{down} 子集中的金属线与 $s_{(3+1) \bmod 4}^{\text{left}} = s_0^{\text{left}}$ 子集中的金属线通过一定规则连接。
- d) $(s_i^{\text{up}}, s_i^{\text{down}})$: 例如, 如图 3 所示, 当 $i=2$ 时, 配置 SB, 可以使得 s_2^{up} 子集中的金属线与 s_2^{down} 子集中的金属线按照一定规则连接起来。
- e) $(s_i^{\text{up}}, s_{(i+1) \bmod 4}^{\text{right}})$: 例如, 如图 3 所示, 当 $i=0$ 时, 配置 SB, 可以使得 s_0^{up} 子集中的金属线与 $s_{(0+1) \bmod 4}^{\text{right}} = s_1^{\text{right}}$ 子集中的金属线通过一定规则连接。
- f) $(s_i^{\text{right}}, s_{(i+1) \bmod 4}^{\text{down}})$: 例如, 如图 3 所示, 当 $i=1$ 时, 配置 SB, 可以使得 s_1^{right} 子集中的金属线与 $s_{(1+1) \bmod 4}^{\text{down}} = s_2^{\text{down}}$ 子集中的金属线通过一定规则连接。

根据如上规则, 我们可以发现通过配置 SB, 可以使得由 s_0^{left} 子集中的金属线进入 SB 的信号, 最多从 s_1^{up} 、 s_0^{right} 和 s_3^{down} 这三个不同方向上的子集中的金属线输出; 使得 s_1^{right} 子集中的金属线最多连接到 s_0^{up} 、 s_1^{left} 和 s_2^{down} 这三个不同方向上的子集中的金属线。

- 通过配置 SB, 对于已经定义好的两个能够连接的子集对, 金属线按照序号由小到大能够完成一一对应连接。例如对于子集对 $(s_0^{\text{left}}, s_3^{\text{down}})$, s_0^{left} 子集中的金属线 $m_0, m_1, \dots, m_7$ 可分别与 s_3^{down} 子集中的金属线 $m_{24}, m_{25}, \dots, m_{31}$ 进行对应连接 (也可以不相连)。

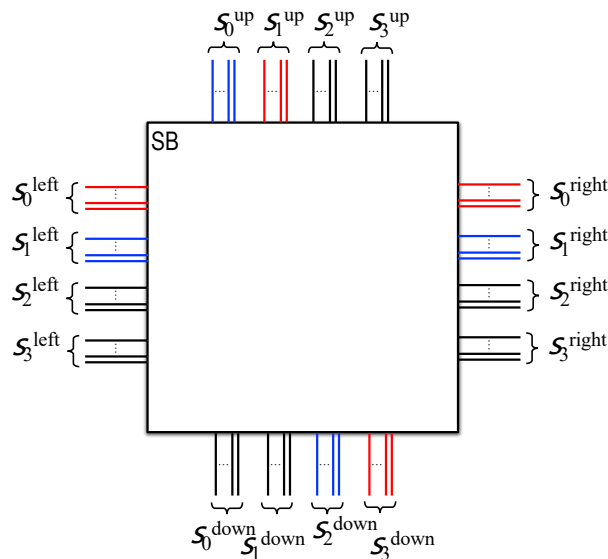


图 3、SB 的互连关系示意图。

2. 网表介绍:

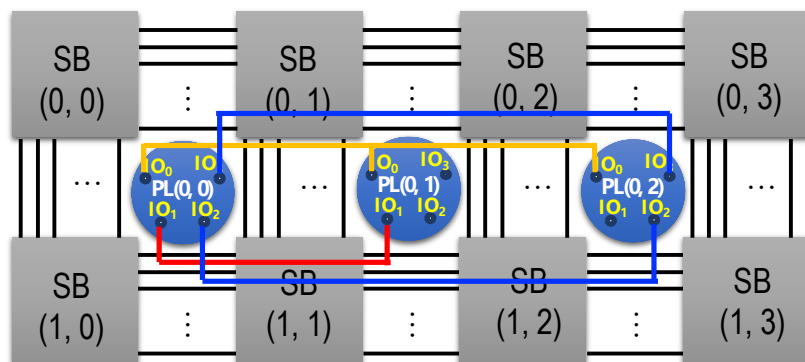
简化起见, 假定在应用 FPGA 时, PL 的配置已经完成, 我们需要根据布线网表确定所有 SB 的配置信息, 以完成不同 PL 之间的连接, 即利用可编程互连资源完成布线。布线网表中通常包含多个线网, 每个线网包含两个或多个端点, 描述了 PL 的 IO 之间的逻辑连接关系, 即需要通过对 PL 和 SB 的配置, 利用单倍

线将同一个线网中的所有端点连接起来。通过对 PL 进行配置，将端点所对应的 IO 连接到 PL 四周的单倍线上；通过对 SB 进行配置，可以完成该 SB 四周单倍线的相互连接。

根据所包含端点的不同，我们可以将所有线网分为三类：

- 两端线网：每一个两端线网仅包含两个端点，这两个端点均为 PL 的 IO 端口，没有时序要求；
- 多端线网：每一个多端线网包含多个（多余两个）端点，这些端点均为 PL 的 IO 端口，没有时序要求；
- bus 线网：每一组 bus 线网包含多个两端线网，这些端点均为 PL 的 IO 端口，要求这一组 bus 线网中的所有两端线网同步（即经过的路径长度相同，或者说每一条路径经过的互连金属线、SB 数量相同），即保证这些两端线网的时延相同。

图 4（a）给出了以上三种线网的示意图，红色路径为一个两端线网；黄色路径是一个多端线网；蓝色路径表示一个 bus 线网组。图 4（b）给出了这三个线网在网表文件中的格式说明（请参考 input_netlist_example.txt）。注意，这里的布线网表描述的是不同 PL 的不同 IO 之间的逻辑连接关系（即哪些端点应该连接在一起），但是并没有给出实际布线，即没有给出实际上两个端点或者多个端点之间应该通过哪些单倍线和 SB 完成连接。



(a)

```
// 按顺序依次定义两端线网、多端线网和bus线网组
1 // 本网表共含有1个两端线网(红色线网)，若没有两端线网，此处应为0
001011 // 第1-2个数字 (0, 0) 表示第一个端点所在的PL的坐标，第3个数字2表示该PL的端口序号为1
// 第4-5个数字 (0, 1) 表示第二个端点所在的PL的坐标，第3个数字1表示该PL的端口序号为1
// 每三个数字可以确定一个PL的IO端口（即线网的一个端点）：第1-2个数字是PL的坐标，第3个数字是PL的IO端口序号
// 每一行表示一个两端线网

1 // 本网表共含有1个多端线网(黄色线网)，若没有多端线网，此处应为0
3 // 以下每个线网包含3个端点
000010020 // 第一个端点是坐标为 (0,0) 的PL上序号为0的IO端口
// 第二个端点是坐标为 (0,1) 的PL上序号为0的IO端口
// 第三个端点是坐标为 (0,2) 的PL上序号为0的IO端口

1 // bus线网组的总数，若没有bus线网，此处应为0
02 // 编号为0的bus线网组共包含2个两端线网
002022 // 编号为0的bus线网组中的第一个两端线网
003023 // 编号为0的bus线网组中的第二个两端线网
```

(b)

图 4、(a) 三类线网示意图；(b) 网表说明示例，注意所有网表都是先给出两端线网，再给出多端线网，最后给出 bus 线网组。此外，图中 “//” 表示单行注释开始。

3. 算法描述：

这一部分我们给出一些常见的布线算法介绍以供参考，可基于这些算法进行改进，也可利用其他算法，以实现本题的编程要求。

3.1.1. 广度优先搜索算法

广度优先搜索（BFS）是一种图搜索算法，用于从给定的起始节点开始，逐层地探索图中的节点，直到找到目标节点或遍历完整个图。下面是广度优先搜索算法的一般描述：

1. 初始化：选择一个起始节点，并将其标记为已访问。
2. 创建一个队列，并将起始节点加入队列。
3. 重复执行以下步骤，直到队列为空：
 - a) 从队列中取出一个节点，将其称为当前节点。
 - b) 检查当前节点的相邻节点（即与当前节点直接相连的节点）：如果相邻节点未被访问过，将其标记为已访问，并将其加入队列。
 - c) 标记当前节点为已处理。
4. 当队列为空或者到达终点时，搜索结束。

广度优先搜索算法的关键在于使用队列来保存待处理的节点。起始节点首先加入队列，然后逐个取出队列中的节点进行处理。通过按照节点的层级顺序处理，即先处理与起始节点直接相连的节点，然后是与这些节点相连的节点，依此类推，BFS 可以保证找到的路径是最短的。在实际应用中，可以使用额外的数据结构来记录节点的访问状态和路径信息，例如标记数组或字典。通过记录每个节点的父节点，可以在搜索完成后从目标节点回溯到起始节点，从而还原出找到的路径。总结来说，广度优先搜索算法通过逐层扩展搜索的方式，从起始节点开始逐步向外扩展，直到找到目标节点或遍历完整个图。它适用于寻找最短路径或层级关系的问题，如图的连通性、最短路径、拓扑排序等。

对于本题的在当前的结构中，我们可以将相邻 SB 之间的金属线和 PL 的 IO 端口都看作图中的可访问节点。对于两端线网，BFS 搜索过程可以描述为：我们将线网的一个端点视为起始节点，然后按照顺序访问与该顶点相连的金属线，通过不断向外扩展，最终找到另一个端点，从而获得最短路径。

3.1.2. A*算法

A*算法是一种启发式搜索算法，常用于解决图搜索和路径规划问题。它综合了广度优先搜索和启发式评估函数的特点，以找到最优路径。A*算法使用两个集合 `open_set` 和 `close_set` 来分别表示待遍历的节点和已经遍历过的节点，具体算法流程如下：

1. 初始化 `open_set` 和 `close_set`；
2. 将起始节点加入 `open_set` 中，并设置优先级为 0（表示优先级最高）；
3. 如果 `open_set` 不为空，则从 `open_set` 中选取优先级最高的节点 n ：
 - a) 如果节点 n 为终点，则从 n 开始逐步反向追踪 `parent` 节点，一直达到起始节点。返回找到的结果路径，算法结束；
 - b) 如果节点 n 不是终点，则将节点 n 从 `open_set` 中删除，将其加入 `close_set` 中；然后遍历节点 n 所有的邻近节点：
 - i. 如果邻近节点 m 在 `close_set` 中，则跳过节点 m ，考察下一个邻近节点；
 - ii. 如果在 `close_set` 和 `open_set` 中都没有邻近节点 m ，则设置节点 m 的 `parent` 为节点 n ，计算 m 的优先级，并将其加入 `open_set` 中；
 - iii. 如果在 `close_set` 中没有节点 m ，但是 `open_set` 中存在，则比较从起始节点到当前节点 n 的路径代价加上当前节点 n 到相邻节点 m 的实际代价与该相邻节点 m 的路径代价。如果新的路径

代价更小，则更新该相邻节点 m 的路径代价，并将 m 的父节点更新为 n 。

c) 根据节点的优先级对 `open_set` 进行排序。

上述算法中，节点 n 的优先级 $f(n)$ 定义如下：

$$f(n) = g(n) + h(n)$$

其中， $g(n)$ 是节点 n 的路径代价 (Path Cost)，指的是从起始节点到当前节点 n 的实际代价或距离。路径代价可以是实际的距离、时间、成本或任何适用的度量标准。 $h(n)$ 表示节点 n 距离终点的启发式评估 (Heuristic Evaluation)，用于估计从当前节点 n 到终点的剩余距离或代价，帮助算法决定搜索方向，以便更有可能找到最优路径。启发式评估函数通常基于问题的特定知识，使得算法更倾向于朝着目标节点的方向搜索。在 A* 算法中，节点的启发式评估值用于衡量从当前节点到目标节点的预估距离或代价。

A* 算法的效率关键在于如何定义优先级，尤其是如何针对图的结构给出准确且高效的启发式评估函数。广度优先算法实际上可认为是 $h(n) = 0$ 的一种较差的 A* 算法。

3.1.3. Dijkstra 算法

Dijkstra 算法是一种图搜索算法，主要用于求解有权图中最短路径问题，可以得到图中一个顶点到其余各顶点的最短路径，注意该算法要求图中不存在负权边。Dijkstra 算法主要特点是从起始点开始，采用贪心算法的策略，每次遍历到始点距离最近且未访问过的顶点的邻接节点，直到扩展到终点为止。

Dijkstra 算法维护两个集合 `open_set` 和 `close_set`，`open_set` 存储还未确定“从起始节点到达该节点的最短路径”（以下简称为最短路径）的节点，`close_set` 存储已经确定了最短路径的节点，具体算法流程如下：

1. 初始化 `open_set` 和 `close_set`，`close_set` 初始为空集，`open_set` 包含图中的所有节点，每个节点均存储父节点和最短路径长度两个信息，并且所有节点的最短路径长度均初始化为无穷大；
2. 将起始节点加入 `close_set` 中，记录其最短路径长度为 0，表示已知从起始节点到达该节点的最短路径为 0（即自身）；
3. 针对于新加入 `close_set` 的节点 n ，遍历其可到达的所有相邻节点：
 - a) 如果该相邻节点在 `close_set` 中，则跳过；
 - b) 如果该相邻节点在 `open_set` 中，则：
 - i. 计算从节点 n 经过有权边到达该相邻节点的最短路径长度 l ；
 - ii. 如果 l 小于该相邻节点当前存储的最短路径长度，则更新该相邻节点的父节点为 n 、最短路径长度为 l ，否则不更新；
4. 遍历更新完成后，在 `open_set` 中选择当前最短路径长度最小的节点，将其加入 `close_set`，视为节点 n ，回到第 3 步。
5. 直到所有节点均在 `close_set` 中，即得到从起始节点到其余各顶点的最短路径

Dijkstra 算法采用贪心算法的策略，不断地选择路径最短的子路径，最终求解得到从起始节点到达所有节点的最短路径，其特点在于求解得到了图中起点到所有节点的最短路径，但需要注意该算法要求图中不存在负权边。

4. 参考文献：

- [1] Peyer, Sven, Dieter Rautenbach, and Jens Vygen. "A generalization of Dijkstra's shortest path algorithm with applications to VLSI routing." *Journal of Discrete Algorithms* 7.4 (2009): 377-390.
- [2] McMurchie, Larry, and Carl Ebeling. "PathFinder: A negotiation-based performance-driven router for FPGAs."

- [3] Ozdal, Muhammet Mustafa, Martin DF Wong, and Philip S. Honsinger. "Simultaneous escape-routing algorithms for via minimization of high-speed boards." *IEEE transactions on computer-aided design of integrated circuits and systems* 27.1 (2007): 84-95.

5. 第一题：两端、多端及 Bus 线网的布线

5.1. 编程要求

- 采用已有算法实现多个两端/多端线网的布线，并尽可能使得总线长最短（即单个网表中的所有线网所经过的所有金属线和所有 SB 数目最少），按要求输出网表中每个线网的布线结果；
- 对布线算法进行改进，实现包含 bus 线网组的网表的布线，保证每一个 bus 线网组内部的两端线网同步，并尽可能使得所有线网的总线长最短（包括两端/多端线网和 bus 线网组），按要求输出网表中每个线网的布线结果；
- 改进布线算法，尽量保证布线资源的均匀利用，以提高布通率；同时提高计算效率。对于每一个 test bench，应给出所有单倍线（每两个相邻 SB 之间的 32 根金属线构成一组单倍线）的利用率（即有多少根线真正在布线中得到应用）和总的布线时间，以评估程序。

5.2. Test Bench（布线网表）

- input_netlist_example.txt: 布线网表输入格式说明。
- input_netlist_tb_demo.txt: 包含 5 个两端线网。
- input_netlist_tb_0.txt: 包含 1 个两端线网，1 个多端线网，1 个 bus 线网组。
- input_netlist_tb_1.txt: 包含 100 个两端线网。
- input_netlist_tb_2.txt: 包含 100 个两端线网和 15 个多端线网。
- input_netlist_tb_3.txt: 包含 100 个两端线网、10 个多端线网、10 个 bus 线网组。
- output_example.txt: 输出格式说明。

5.3. 输出文件格式

针对每一个 test bench，输出文件需给出其中包含的每个网表的布线结果（如图 5（a）所示，其中给出了图 4 中两端线网和多端线网的布线结果），以及编程要求中提到的利用率、布线时间、线网总长等统计信息（如图 5（b）所示）（请参考 output_example.txt）。例如，图 5（a）中表示图 4（a）中的两端线网从位于 (0,0) 处的 PL 的 1 号 IO 出发，连接位于 (0,1) 的 SB 的左侧第 8 根金属线，再连接位于 (0,2) 的 SB 的左侧第 8 根金属线，然后进入位于 (0,1) 处的 PL 的 1 号 IO。然后，我们就可以根据该输出文件配置 PL 和 SB 了。

```
// 两端线网输出格式
// 按输入的线网顺序先给出线网序号
two_end_0 // 两端线网中的0号线网
IO_0 0 0 1 // 第一个端点坐标，同输入
wire 0 1 0 8 // 金属线坐标，该行表示 (0, 1) SB左侧的8号金属线
wire 0 2 0 8 // 金属线坐标，该行表示 (0, 2) SB左侧的8号金属线
IO_1 0 1 1 // 第二个端点坐标，同输入
length: 2 // 该线网路径长度，按照金属线数量计数

|
// 多端线网输出
// 按输入的线网顺序先给出线网序号
// 依次输出第一个端点到其他端点的路径（第一到第二，第一到第三，第一到第四.....）
mul_end_0 // 多端线网中的0号线网
IO_0 0 0 0 // 第一个端点所在的PL的坐标，同输入
wire 0 1 0 0 // 金属线坐标，该行表示 (0, 1) SB左侧的0号金属线
wire 0 2 0 0 // 金属线坐标，该行表示 (0, 2) SB左侧的0号金属线
IO_1 0 1 0 // 第二个端点所在的PL的坐标，同输入
length: 2 // 该线网路径长度，按照金属线数量计数

IO_0 0 0 0 // 第一个端点所在的PL的坐标，同输入
wire 0 1 0 0 // 金属线坐标，该行表示 (0, 1) SB左侧的0号金属线
wire 0 2 0 0 // 金属线坐标，该行表示 (0, 2) SB左侧的0号金属线
wire 0 3 0 0 // 金属线坐标，该行表示 (0, 3) SB左侧的0号金属线
IO_2 0 2 0 // 第三个端点所在的PL的坐标，同输入
length: 3 // 该线网路径长度，按照金属线数量计数
```

(a)

```
utilization // 当前开始输出利用率
wire group 0 1 0 0.5313 // 金属线组坐标，该行表示 (0, 1) SB左侧的一组金属线，使用率0.5313，保留四位小数，即17/32
wire group 0 2 0 0.3125 // 金属线组坐标，该行表示 (0, 2) SB左侧的一组金属线，使用率0.3125，保留四位小数，即10/32
wire group 0 3 0 0.0000
.....
wire group 0 15 0 0.0000
wire group 1 0 1 0.0000 // 金属线组坐标，该行表示 (1, 0) SB上侧的一组金属线，使用率0.0000，保留四位小数，即0/32
wire group 1 1 0 0.0000
wire group 1 1 1 0.0000
wire group 1 2 0 0.0000
wire group 1 2 1 0.0000
wire group 1 3 0 0.0000
wire group 1 3 1 0.0000
.....
wire group 15 15 0 0.0000
wire group 15 15 1 0.0000

max 0.5313 // 最大利用率为0.5313，保留四位小数，即17/32
min 0.0000 // 最大利用率为0.5313，保留四位小数，即0/32
avg 0.0020 // 平均利用率，即420组金属线的平均利用率
time 100 s // 布线时间，利用程序时间统计，单位为秒
length 2023 // 所有线网的长度求和
```

(b)

图 5、(a) 线网路径输出格式示例；(b) 统计信息输出格式示例。

6. 第二题：多个网表的增量式布线

FPGA 应用中，有时我们需要在已完成某个网表布线的前提下，改变尽可能少的布线资源的配置，以尽快完成一组新的线网布线。这类布线我们称为增量式布线，目标是最小化需改变配置的布线资源的数目。

6.1. 编程要求

- 在 test bench 中，对于第一题中的每一个网表，均有一个相对应的改变后的网表（例如，对应于第一题中的网表 input_netlist_tb_demo.txt，我们改变了其中的 2 个两端线网，生成了一个新的网表 edit_tb_demo.txt）。增量式布线的要求是利用 input_netlist_tb_demo.txt 的布线结果，做尽量少的变化，完成新的网表 edit_tb_demo.txt 的布线。
- 对布线算法进行改进，在尽量保证线网的布线总线长最短的前提下，使得在从第一个网表布线改为第二个网表时，所需改变的布线资源最少，并按要求输出布线结果以及改变的布线资源数目；

- 改进布线算法，提高计算效率。对于每一个 test bench，应给出从第一个网表转变为第二个网表时的计算时间，以评估程序。

6.2. Test Bench（布线网表）

- edit_tb_demo.txt：对应 input_netlist_tb_demo.txt：包含 5 个两端线网；其中 2 个两端线网发生了变化。
- edit_tb_0.txt：对应 input_netlist_tb_0.txt，包含 1 个两端线网，1 个多端线网，1 个 bus 线网组；其中 1 个多端网发生了变化。
- edit_tb_1.txt：对应 input_netlist_tb_1.txt，包含 100 个两端线网；其中 10 个两端线网发生了变化。
- edit_tb_2.txt：对应 input_netlist_tb_2.txt，包含 100 个两端线网和 15 个多端线网；其中 10 个两端线网、3 个多端线网发生了变化。
- edit_tb_3.txt：对应 input_netlist_tb_3.txt，包含 100 个两端线网、10 个多端线网、10 个 bus 线网组；其中 10 个两端线网、3 个多端线网、1 个同步线网组发生了变化。
- edit_output_example.txt：输出格式说明。

6.3. 输出文件格式

针对每一个新的网表，输出文件首先需要给出如第一题图 5 所示的新的布线结果，然后给出如图 6 所示的需要做出改变的布线资源数目，以及修改布线所需的计算时间（即修改布线的程序的运行时间）。在本题中改变的布线资源数目仅统计金属线即可。只要某一金属线使用情况在两次布线中发生变化（从使用变为未使用，或者从未使用变为使用），即统计一次，额外输出信息示例如图 6 所示（请参考 edit_output_example.txt）：

```
// 第二题额外输出信息
// 如果某一金属线使用情况在两次布线中发生变化（从使用变为未使用，或者从未使用变为使用），即统计一次
altered wires 123      //123根金属线的使用情况发生变化
edit time    20 s      //修改线网所用时间，利用程序时间统计，单位为秒
```

图 6、增量式布线额外输出信息示例。